

Rapport de TP – Simulateur à événements discrets

Séance 1/3 : architecture, messages, événements, ordonnanceur et traces

Groupe 3

PINCAUD Steve, SAVIARD Matthieu et CHARONNET Henri

6 mai 2026

Résumé

Ce rapport présente le travail réalisé lors de la première séance du TP consacré à la conception d'un simulateur à événements discrets dans le cadre de l'évaluation des performances des systèmes informatiques. Cette première séance ne vise pas encore à simuler complètement une file d'attente, mais à construire les fondations logicielles indispensables : interfaces, messages, événements, ordonnanceur, moteur principal, tests et traces.

L'objectif principal est de préparer progressivement un simulateur orienté objet capable, lors des séances suivantes, de modéliser des systèmes de files d'attente de type M/M/1, M/M/1/K et M/M/c/K. Le rapport analyse en détail les choix de conception, les conventions de nommage, les méthodes de chaque classe, la stratégie de test et la configuration du fichier de log `tp2.log`. Une attention particulière est portée aux fichiers `*Impl.py` et au fichier `Engine.py`, car ils constituent le cœur observable de l'implémentation.

Table des matières

1 Introduction

La simulation à événements discrets est une méthode fondamentale pour étudier le comportement dynamique de systèmes informatiques. Elle est particulièrement adaptée à l'évaluation de performances, car elle permet de représenter l'évolution d'un système uniquement aux instants où un changement d'état se produit.

Contrairement à une simulation à pas de temps fixe, dans laquelle le temps avance régulièrement même lorsque rien ne se passe, une simulation à événements discrets saute directement d'un événement au suivant. Cette approche est donc plus efficace et plus naturelle pour représenter des systèmes tels que :

- des files d'attente ;
- des serveurs de traitement ;
- des passerelles IoT ;
- des systèmes client-serveur ;
- des réseaux de communication.

Dans le cadre du TP, le système étudié repose sur un modèle progressivement construit autour d'un client, d'une passerelle, d'une file d'attente et d'un ou plusieurs serveurs. La séance 1 se concentre sur les composants fondamentaux du simulateur :

- la classe `MessageImpl`, qui représente un message échangé dans le système ;
- la classe `EventImpl`, qui représente un événement discret associé à un instant donné ;
- la classe `SchedulerImpl`, qui maintient la liste des événements futurs dans l'ordre chronologique ;
- le fichier `Engine.py`, qui jouera le rôle de moteur principal ;
- les fichiers de test permettant de vérifier chaque composant ;
- le système de journalisation permettant de produire une trace d'exécution.

Les événements manipulés à cette étape sont :

- `SEND_MSG` : un client envoie un message ;
- `RECV_MSG` : la passerelle reçoit le message ;
- `MSG_DEPT` : le message quitte le système après traitement.

Cette première séance est donc essentiellement une séance d'architecture et de préparation. Elle pose les bases nécessaires pour les séances suivantes, qui introduiront les clients, les files, les serveurs, la passerelle et les métriques de performance.

2 Objectifs de la séance 1

D'après le sujet du TP, la séance 1 a pour objectif de construire les premiers composants du simulateur. Elle se concentre sur :

- le `Main` et le moteur d'exécution ;
- la génération de traces ;
- les méthodes de test ;
- la classe `Message` ;
- la classe `Event` ;
- la classe `Scheduler`.

L'objectif n'est donc pas encore d'obtenir des métriques de performance complètes, mais de valider les briques techniques qui permettront ensuite de produire ces métriques.

2.1 Travail attendu

Le travail attendu pendant cette séance peut se résumer en quatre axes.

Premier axe : modélisation des données. Il faut représenter les messages et les événements à l'aide de classes dédiées. Ces classes doivent contenir les informations nécessaires au suivi temporel du système.

Deuxième axe : organisation temporelle. Il faut créer un ordonnanceur capable de stocker les événements futurs et de les restituer dans l'ordre chronologique.

Troisième axe : testabilité. Chaque classe doit pouvoir être testée indépendamment. Cette approche correspond à une méthodologie incrémentale : on valide une brique avant de construire la suivante.

Quatrième axe : traçabilité. Le simulateur doit produire des traces permettant de vérifier l'ordre des événements, de déboguer le code et, plus tard, de calculer des métriques a posteriori.

3 Prérequis et environnement d'exécution

3.1 Langage utilisé

Le TP est développé en Python 3. Le choix de Python est adapté à ce type de TP pour plusieurs raisons :

- syntaxe concise ;
- facilité de prototypage ;
- disponibilité de bibliothèques scientifiques ;
- compatibilité avec les tests unitaires ;
- bonne lisibilité du code orienté objet.

3.2 Organisation du projet

Le projet est organisé dans l'arborescence suivante :

```
rcp103/  
|-- tp2.log  
|-- net/  
    |-- lecnam/  
        |-- rcp103/  
            |-- tp2/  
                |-- Engine.py  
                |-- EventImpl.py  
                |-- EventImplMockTest.py  
                |-- EventImplTest.py  
                |-- EventType.py  
                |-- IEvent.py  
                |-- IMessage.py  
                |-- IScheduler.py  
                |-- Message.py  
                |-- MessageImpl.py  
                |-- MessageImplTest.py  
                |-- SchedulerImpl.py  
                |-- logging_config.cnf
```

Cette organisation sous forme de packages Python permet d'utiliser des imports absolus du type :

```
1 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
```

Listing 1 – Exemple d'import absolu

3.3 Commandes d'exécution

Les fichiers de test peuvent être lancés depuis la racine du projet. Par exemple :

```
1 PYTHONPATH=. python3 net/lecnam/rcp103/tp2/MessageImplTest.py  
2 PYTHONPATH=. python3 net/lecnam/rcp103/tp2/EventImplTest.py  
3 PYTHONPATH=. python3 net/lecnam/rcp103/tp2/EventImplMockTest.py
```

```
4 PYTHONPATH=. python3 net/lecnam/rcp103/tp2/Engine.py
```

Listing 2 – Exécution des tests sous Linux/WSL

Il est également possible d'utiliser l'option `-m` de Python :

```
1 python3 -m net.lecnam.rcp103.tp2.MessageImplTest
2 python3 -m net.lecnam.rcp103.tp2.EventImplTest
3 python3 -m net.lecnam.rcp103.tp2.Engine
```

Listing 3 – Exécution sous forme de module

Sous Windows, les fichiers peuvent être exécutés depuis PowerShell avec :

```
1 python MessageImplTest.py
2 python EventImplTest.py
3 python Engine.py
```

Listing 4 – Exécution sous Windows

3.4 Installation des bibliothèques

Le fichier `EventImpl.py` contient plusieurs commentaires rappelant les prérequis d'installation. Même si toutes les bibliothèques ne sont pas encore pleinement exploitées pendant la séance 1, elles préparent les séances suivantes.

Les bibliothèques standards utilisées sont notamment :

- `os`, pour la manipulation des chemins ;
- `sys`, pour l'environnement d'exécution ;
- `logging` et `logging.config`, pour la journalisation ;
- `traceback`, pour l'affichage détaillé des erreurs ;
- `datetime`, pour la manipulation éventuelle d'horodatages ;
- `enum`, pour la définition des types d'événements.

Les bibliothèques scientifiques importées sont :

- `numpy` ;
- `scipy` ;
- `matplotlib`.

Sous Linux ou WSL, les commandes suivantes peuvent être utilisées :

```
1 sudo apt update
2 sudo apt upgrade
3 sudo apt install python3-pip
4 sudo apt install python3-scipy
5 sudo apt install python3-matplotlib
6
7 python3 -m pip install -U pip
8 python3 -m pip install -U numpy
9 python3 -m pip install -U scipy
10 python3 -m pip install -U matplotlib
```

Listing 5 – Installation des bibliothèques sous Linux/WSL

Sous Windows, si Python est installé dans `c:\python314`, les commandes peuvent être :

```
1 c:\python314\python.exe -m pip install -U pip
2 c:\python314\python.exe -m pip install numpy
3 c:\python314\python.exe -m pip install scipy
4 c:\python314\python.exe -m pip install matplotlib
```

Listing 6 – Installation des bibliothèques sous Windows

Pour vérifier les bibliothèques installées :

```
1 pip list
```

Listing 7 – Vérification des bibliothèques

Dans Visual Studio Code, il est conseillé d'utiliser l'extension *Microsoft Python Environments Extension*. En cas de problème de détection d'environnement, il est possible d'exécuter depuis la palette de commandes :

Python Environments: Refresh All Environment Managers

4 Conventions de nommage

4.1 Conventions générales

Le projet suit plusieurs conventions de nommage importantes. Ces conventions permettent de rendre le code plus lisible et de clarifier le rôle de chaque fichier.

4.2 Interfaces : préfixe I

Les fichiers commençant par I représentent des interfaces abstraites :

- `IMessage.py`;
- `IEvent.py`;
- `IScheduler.py`.

Cette convention est classique dans les architectures orientées objet. Elle indique que le fichier ne contient pas nécessairement une logique métier complète, mais plutôt un contrat que les classes concrètes devront respecter.

4.3 Implémentations : suffixe Impl

Les fichiers suffixés par Impl représentent les implémentations concrètes :

- `MessageImpl.py`;
- `EventImpl.py`;
- `SchedulerImpl.py`.

Ces fichiers sont particulièrement importants, car ils constituent le cœur fonctionnel de la séance 1. Ce sont eux que l'enseignant est susceptible d'examiner prioritairement pour vérifier :

- la cohérence avec les interfaces ;
- la qualité de l'encapsulation ;
- la lisibilité du code ;
- la présence des méthodes attendues ;
- la conformité avec le sujet.

4.4 Tests : suffixe Test

Les fichiers terminés par `Test.py` contiennent les tests associés :

- `MessageImplTest.py`;
- `EventImplTest.py`;
- `EventImplMockTest.py`.

Cette convention permet d'associer facilement chaque classe à son test. Elle s'inscrit dans une démarche de développement incrémental.

4.5 Moteur principal : Engine.py

Le fichier `Engine.py` correspond au point d'entrée du programme. Il contient notamment le bloc :

```
1 if __name__ == "__main__":  
2     engine = Engine()  
3     engine.create_clients(3)  
4     engine.run_tests()
```

Listing 8 – Point d'entrée Python

Ce fichier sera amené à devenir le moteur principal du simulateur. Même si son rôle est encore limité en séance 1, il est essentiel pour structurer les futures simulations.

5 Architecture générale du simulateur

5.1 Principe d'un simulateur à événements discrets

Un simulateur à événements discrets fonctionne autour d'une liste d'événements futurs. Chaque événement possède un temps d'occurrence. Le simulateur répète ensuite les étapes suivantes :

1. extraire l'événement le plus proche dans le temps ;
2. avancer le temps courant jusqu'au temps de cet événement ;
3. traiter l'événement ;
4. modifier l'état du système ;
5. programmer éventuellement de nouveaux événements ;
6. enregistrer une trace.

Cette logique permet d'éviter de simuler les périodes durant lesquelles aucun changement ne se produit.

5.2 Responsabilités des composants

L'architecture de la séance 1 peut être résumée ainsi :

- `MessageImpl` : représente les messages transportés dans le système ;
- `EventImpl` : représente les événements horodatés ;
- `EventType` : définit les types d'événements possibles ;
- `SchedulerImpl` : maintient les événements dans l'ordre chronologique ;
- `Engine` : orchestre les tests et prépare la future boucle de simulation ;
- `logging.config.cnf` : configure les traces d'exécution.

5.3 Intérêt de la séparation interface / implémentation

La séparation entre interfaces et implémentations présente plusieurs avantages :

- elle clarifie les responsabilités ;
- elle facilite le remplacement d'une implémentation ;
- elle rend le code plus testable ;
- elle prépare l'utilisation de mocks ;
- elle rapproche le projet d'une architecture logicielle industrielle.

D'un point de vue conception, cette séparation s'inscrit dans les principes SOLID, notamment le principe d'inversion de dépendance : les composants doivent dépendre d'abstractions plutôt que de classes concrètes.

6 Analyse des interfaces

6.1 Interface IMessage

L'interface `IMessage` représente le contrat minimal d'un message. Un message doit pouvoir fournir :

- son identifiant ;
- sa source ;
- sa destination ;
- son timestamp ;
- une représentation textuelle.

Même si le fichier transmis est minimal, son existence structure l'architecture : toute classe représentant un message doit respecter ce contrat.

6.2 Interface IEvent

L'interface `IEvent` représente le contrat d'un événement. Un événement doit contenir un horodatage, un type et un message associé. Il doit aussi être affichable pour permettre la génération de traces.

6.3 Interface IScheduler

L'interface `IScheduler` contient au minimum la méthode `add_event`. Cette méthode définit la responsabilité principale de l'ordonnanceur : ajouter un événement à la liste des événements futurs.

```
1 class IScheduler(ABC):
2     @abstractmethod
3     def add_event(self, event: IEvent):
4         pass
```

Listing 9 – Extrait de `IScheduler`

Cette interface sera complétée naturellement par des méthodes comme :

- `get_event()` ;
- `get_current_time()` ;
- `has_events()` ;
- `print_scheduler()`.

7 Analyse détaillée de `MessageImpl`

7.1 Rôle de la classe

La classe `MessageImpl` représente un message envoyé vers la passerelle. Elle est une brique fondamentale, car tous les événements `SEND_MSG`, `RECV_MSG` et `MSG_DEPT` manipulent un message.

Un message contient :

- `message_id` : identifiant unique ;
- `source` : entité émettrice ;
- `destination` : entité destinataire ;
- `timestamp` : temps de création.

Le timestamp est essentiel pour calculer ultérieurement le temps de séjour :

$$W_{\text{système}} = t_{\text{départ}} - t_{\text{création}}$$

7.2 Constructeur

```
1 def __init__(self, message_id, source, destination, timestamp=0.0):
2     self._message_id = message_id
3     self._source = source
4     self._destination = destination
5     self._timestamp = timestamp
```

Listing 10 – Constructeur de MessageImpl

Le constructeur initialise les quatre informations fondamentales du message.

Analyse. La valeur par défaut `timestamp=0.0` permet de créer un message même lorsque le temps courant n'a pas encore été explicitement défini. Dans une simulation complète, il serait préférable d'affecter systématiquement le timestamp au temps courant du scheduler.

7.3 Méthode `get_message_id`

```
1 def get_message_id(self):
2     return self._message_id
```

Cette méthode retourne l'identifiant du message. Cet identifiant est nécessaire pour suivre un message dans les traces et vérifier la cohérence entre événements.

7.4 Méthode `get_source`

```
1 def get_source(self):
2     return self._source
```

Cette méthode retourne la source du message. Elle sera utile pour identifier le client émetteur.

7.5 Méthode `get_destination`

```
1 def get_destination(self):
2     return self._destination
```

Cette méthode retourne la destination du message. Dans le modèle cible, il s'agira généralement de la passerelle.

7.6 Méthode `get_timestamp`

```
1 def get_timestamp(self):
2     return self._timestamp
```

Cette méthode retourne le temps de création du message. Elle sera essentielle pour les calculs de performance.

7.7 Setters

Les setters permettent de modifier les attributs après construction :

```
1 def set_message_id(self, message_id):
2     self._message_id = message_id
3
4 def set_source(self, source):
5     self._source = source
6
7 def set_destination(self, destination):
8     self._destination = destination
9
10 def set_timestamp(self, temps):
11     self._timestamp = temps
```

Listing 11 – Setters de MessageImpl

Analyse critique. Pour un TP pédagogique, ces setters facilitent les tests. Dans une version industrielle, l'identifiant du message pourrait être rendu immuable afin d'éviter des incohérences dans les traces.

7.8 Méthode `print_message`

```
1 def print_message(self):
2     return(f"[Message] ID={self._message_id} "
3           f"src={self._source} "
4           f"dst={self._destination} "
5           f"timestamp={self._timestamp:.4f}")
```

Cette méthode retourne une représentation textuelle du message. Le timestamp est affiché avec quatre décimales, ce qui convient bien à une simulation probabiliste manipulant des temps flottants.

8 Analyse de EventType

8.1 Rôle

`EventType` est une énumération qui définit les types d'événements manipulés par le simulateur :

```
1 from enum import Enum
2
3 class EventType(Enum):
4     SEND_MSG = 1
5     RECV_MSG = 2
6     MSG_DEPT = 3
```

Listing 12 – `EventType.py`

8.2 Analyse des événements

SEND_MSG. Cet événement représente l'émission d'un message par un client. Il marque l'entrée logique d'un message dans le scénario simulé.

RECV_MSG. Cet événement représente la réception du message par la passerelle. À partir de cet instant, le message peut être traité immédiatement ou placé en file d'attente.

MSG_DEPT. Cet événement représente la fin du traitement d'un message. Il permettra de calculer le temps de séjour et de libérer éventuellement le serveur.

8.3 Intérêt de l'énumération

L'utilisation d'une énumération est préférable à l'utilisation de chaînes de caractères libres. Elle permet :

- d'éviter les fautes de frappe ;
- de centraliser les types d'événements ;
- d'améliorer la lisibilité ;
- de rendre le code plus robuste.

Une amélioration possible serait d'utiliser partout :

```
1 EventType.SEND_MSG
```

au lieu de :

```
1 "SEND_MSG"
```

9 Analyse détaillée de EventImpl

9.1 Rôle de la classe

La classe `EventImpl` représente un événement discret du simulateur. Elle relie trois dimensions :

- une dimension temporelle : le temps de l'événement ;
- une dimension fonctionnelle : le type d'événement ;
- une dimension métier : le message associé.

Un événement est donc un objet central dans la simulation. Le scheduler ne manipule pas directement des messages, mais des événements contenant ces messages.

9.2 En-tête et documentation technique

Le fichier `EventImpl.py` contient un en-tête indiquant :

- la commande d'exécution avec `PYTHONPATH=.` ;
- les prérequis liés à Visual Studio Code ;
- les commandes d'installation de bibliothèques ;
- les liens vers `scipy` et `matplotlib`.

Cet en-tête est utile, car il documente directement l'environnement nécessaire au projet.

9.3 Imports standards

```
1 import datetime
2 import os
3 import platform
4 import string
5 import sys
6 import secrets
7 import traceback
8 import logging
9 import logging.config
```

Listing 13 – Imports standards de EventImpl

Ces imports permettent de gérer :

- les chemins de fichiers ;
- la configuration des logs ;
- l'affichage d'erreurs ;
- l'environnement d'exécution.

9.4 Imports scientifiques

```
1 from scipy.stats import poisson
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import scipy
```

Listing 14 – Imports scientifiques

Ces imports sont surtout préparatoires. Ils seront utiles dans les séances suivantes pour générer des variables aléatoires, manipuler des distributions ou tracer des résultats.

9.5 Imports internes au projet

```
1 from net.lecnam.rcp103.tp2.IEvent import IEvent
2 from net.lecnam.rcp103.tp2.IMessage import IMessage
3 from net.lecnam.rcp103.tp2.EventType import EventType
```

Listing 15 – Imports internes

Ces imports montrent que `EventImpl` dépend :

- de l'interface `IEvent` ;
- de l'interface `IMessage` ;
- de l'énumération `EventType`.

9.6 Configuration du logging

Le fichier charge la configuration du logging avec :

```
1 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")
2 logging.config.fileConfig(
3     config_path,
4     defaults=None,
5     disable_existing_loggers=True,
6     encoding=None
7 )
8
9 logger = logging.getLogger(__name__)
```

Listing 16 – Chargement de `logging_config.cnf`

La ligne :

```
1 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")
```

indique que le fichier de configuration est recherché dans le même dossier que `EventImpl.py`. Ce choix rend le chargement de la configuration plus robuste.

9.7 Déclaration de la classe

```
1 class EventImpl(IEvent):
```

Listing 17 – Déclaration de `EventImpl`

Cette déclaration indique que `EventImpl` est une implémentation concrète de l'interface `IEvent`.

9.8 Méthodes attendues

D'après le test `EventImplTest.py`, la classe fournit au minimum :

- un constructeur ;
- `get_event_time()` ;
- `print_event()`.

Le test utilise :

```
1 msg = MessageImpl(1, "Alice", "Bob", 1.13)
2 impl = EventImpl(1, msg, "SEND_MSG", 1.13)
3 horodatage = impl.get_event_time()
4 res = impl.print_event()
```

Listing 18 – Utilisation de `EventImpl`

On en déduit que le constructeur manipule :

- un identifiant ;
- un message ;
- un type d'événement ;
- un horodatage.

9.9 Analyse critique

La classe `EventImpl` doit rester simple. Elle ne doit pas contenir trop de logique métier, car son rôle principal est de représenter un événement. Le traitement des événements devrait plutôt être placé dans le moteur ou la passerelle.

Une amélioration importante serait de remplacer l'utilisation de chaînes comme `"SEND_MSG"` par l'énumération `EventType.SEND_MSG`. Cela renforcerait la robustesse du code.

10 Analyse détaillée de `SchedulerImpl`

10.1 Rôle de l'ordonnanceur

Le scheduler est le cœur d'un simulateur à événements discrets. Sa responsabilité principale est de maintenir la liste des événements futurs triée par temps croissant.

Il doit permettre :

- l'ajout d'un événement ;
- l'extraction du prochain événement ;
- la consultation du temps courant ;
- l'affichage du contenu de la liste d'événements.

10.2 Méthode `add_event`

La méthode `add_event` est définie dans l'interface `IScheduler`. Elle doit ajouter un événement au bon endroit dans la liste.

```
1 def add_event(self, event):
2     i = 0
3     while i < len(self._events) and \
4           self._events[i].get_event_time() <= event.get_event_time():
5         i += 1
6     self._events.insert(i, event)
```

Listing 19 – Principe d'insertion chronologique

Analyse algorithmique. Cette insertion est en complexité $O(n)$. Pour une première version pédagogique, ce choix est acceptable. Pour des simulations plus importantes, une file de priorité avec `heapq` serait plus efficace.

10.3 Méthode `get_event`

```
1 def get_event(self):
2     if not self._events:
3         return None
4     event = self._events.pop(0)
5     self._current_time = event.get_event_time()
6     return event
```

Listing 20 – Principe de `get_event`

Cette méthode extrait l'événement le plus ancien et met à jour le temps courant.

10.4 Méthode `get_current_time`

```
1 def get_current_time(self):
2     return self._current_time
```

Listing 21 – Principe de `get_current_time`

Elle permet aux autres composants de connaître le temps simulé.

10.5 Méthode has_events

```
1 def has_events(self):  
2     return len(self._events) > 0
```

Listing 22 – Principe de *has_events*

Cette méthode est utile dans la boucle principale :

```
1 while scheduler.has_events():  
2     event = scheduler.get_event()  
3     traiter(event)
```

Listing 23 – Boucle principale conceptuelle

10.6 Validation par la trace

La trace `tp2.log` montre les événements suivants :

```
Event ID: 1, Type: SEND_MSG, Time: 1.202  
Event ID: 2, Type: RECV_MSG, Time: 1.916  
Event ID: 3, Type: SEND_MSG, Time: 2.32  
Event ID: 4, Type: RECV_MSG, Time: 2.391  
Event ID: 5, Type: MSG_DEPT, Time: 4.572  
Event ID: 6, Type: MSG_DEPT, Time: 5.916
```

Les temps sont bien ordonnés :

$$1.202 < 1.916 < 2.320 < 2.391 < 4.572 < 5.916$$

Cela valide le rôle principal du scheduler.

11 Analyse de Engine.py

11.1 Rôle général

Le fichier `Engine.py` est le point d'entrée du programme. Dans une version complète du simulateur, il contiendra :

- les paramètres globaux ;
- la création des clients ;
- l'initialisation de la passerelle ;
- l'initialisation du scheduler ;
- la boucle principale de simulation ;
- la génération de traces ;
- le calcul des métriques.

11.2 Chargement du logging

Comme `EventImpl.py`, `Engine.py` charge le fichier `logging_config.cnf` :

```
1 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")  
2 logging.config.fileConfig(config_path, defaults=None,  
3                             disable_existing_loggers=True,  
4                             encoding=None)  
5  
6 logger = logging.getLogger(__name__)
```

Listing 24 – Configuration du logging dans `Engine.py`

Cette configuration garantit que le moteur produit des logs homogènes avec le reste du projet.

11.3 Bloc principal

Le bloc principal est :

```
1 if __name__ == "__main__":  
2     engine = Engine()  
3     engine.create_clients(3)  
4     engine.run_tests()
```

Listing 25 – Main de Engine.py

Ce bloc montre que l’Engine est conçu pour :

- instancier un moteur ;
- créer des clients ;
- lancer les tests.

Même si toutes les méthodes ne sont pas encore détaillées dans la séance 1, cette structure prépare clairement les séances suivantes.

12 Tests réalisés

12.1 Test de MessageImpl

Le test crée un message :

```
1 impl = MessageImpl(1, "Alice", "Bob", 1.21)  
2 src = impl.get_source()  
3 dst = impl.get_destination()  
4 res = impl.print_message()
```

Listing 26 – Création d’un message

Ce test vérifie :

- l’instanciation ;
- l’accès à la source ;
- l’accès à la destination ;
- l’affichage formaté.

12.2 Test de EventImpl

```
1 msg = MessageImpl(1, "Alice", "Bob", 1.13)  
2 impl = EventImpl(1, msg, "SEND_MSG", 1.13)  
3 horodatage = impl.get_event_time()  
4 res = impl.print_event()
```

Listing 27 – Test de EventImpl

Ce test valide :

- l’association entre événement et message ;
- la récupération de l’horodatage ;
- l’affichage de l’événement.

12.3 Test avec mock

Le fichier EventImplMockTest.py utilise MagicMock :

```
1 mock_sim = MagicMock(spec=EventImpl)  
2 mock_sim.get_event_time.return_value = 1.42  
3  
4 result = mock_sim.get_event_time()  
5 mock_sim.get_event_time.assert_called_with()
```

Listing 28 – Utilisation de MagicMock

Analyse. L'utilisation d'un mock permet d'isoler une classe ou une méthode. C'est une pratique intéressante, car les futures séances introduiront davantage d'interactions entre objets. Les mocks permettront de tester un composant sans dépendre de l'implémentation complète des autres composants.

12.4 Limites des tests actuels

Les tests actuels reposent surtout sur des affichages. Ils pourraient être renforcés avec des assertions :

```
1 assert impl.get_source() == "Alice"
2 assert impl.get_destination() == "Bob"
3 assert impl.get_timestamp() == 1.21
```

Listing 29 – Exemple d'assertions

Pour le scheduler, un test utile serait :

```
1 scheduler.add_event(EventImpl(1, msg1, EventType.SEND_MSG, 5.0))
2 scheduler.add_event(EventImpl(2, msg2, EventType.SEND_MSG, 2.0))
3 scheduler.add_event(EventImpl(3, msg3, EventType.SEND_MSG, 3.0))
4
5 assert scheduler.get_event().get_event_time() == 2.0
6 assert scheduler.get_event().get_event_time() == 3.0
7 assert scheduler.get_event().get_event_time() == 5.0
```

Listing 30 – Test conceptuel du scheduler

13 Journalisation et fichier tp2.log

13.1 Rôle de la journalisation

La journalisation permet :

- de vérifier l'ordre des événements ;
- de déboguer le scheduler ;
- de conserver une trace des exécutions ;
- de préparer le calcul futur des métriques.

13.2 Configuration du fichier logging_config.cnf

Le fichier contient :

```
1 [loggers]
2 keys=root
3
4 [handlers]
5 keys=consoleHandler,fileHandler
6
7 [formatters]
8 keys=simpleFormatter
9
10 [logger_root]
11 level=DEBUG
12 handlers=consoleHandler,fileHandler
13
14 [handler_consoleHandler]
15 class=StreamHandler
16 level=DEBUG
17 formatter=simpleFormatter
18 args=(sys.stdout,)
19
20 [handler_fileHandler]
21 class=FileHandler
22 level=DEBUG
23 formatter=simpleFormatter
24 args=('tp2.log', 'a')
```

```

25 [formatter_simpleFormatter]
26 format=%(asctime)s %(levelname)s %(name)s: %(message)s
27
28 datefmt=%Y-%m-%d %H:%M:%S

```

Listing 31 – Configuration du logging

13.3 Chemin du fichier tp2.log

Le fichier de configuration est chargé depuis le dossier contenant les sources :

```

1 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")
2 logging.config.fileConfig(config_path, defaults=None,
3                             disable_existing_loggers=True,
4                             encoding=None)

```

Listing 32 – Chargement de la configuration

En revanche, dans `logging_config.cnf`, la ligne :

```

1 args=('tp2.log', 'a')

```

indique un chemin relatif. Ainsi, le fichier `tp2.log` est créé relativement au répertoire depuis lequel le programme est lancé.

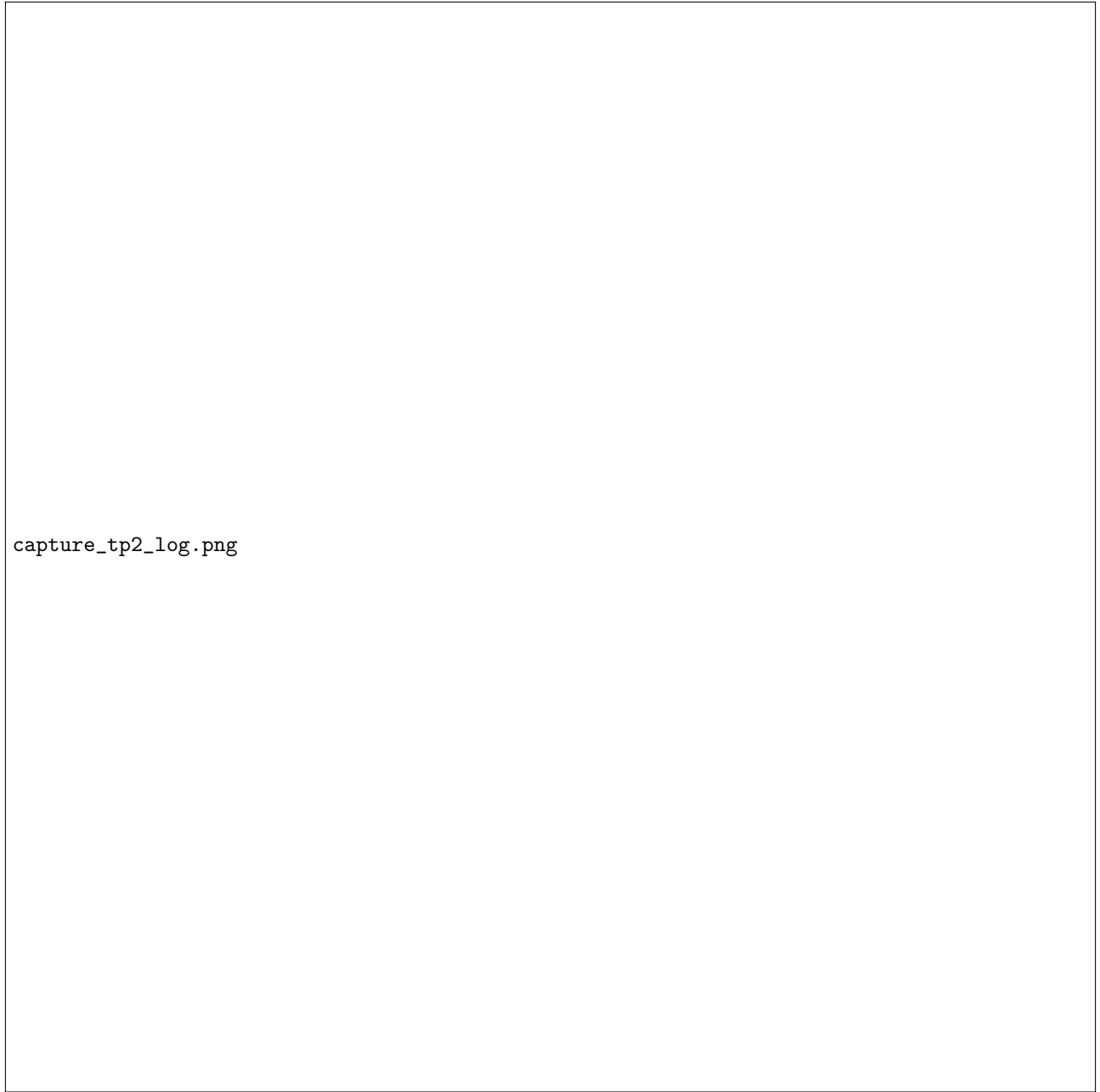
Dans notre cas, le programme est lancé depuis la racine `rcp103`. Le fichier est donc généré à l'emplacement :

`rcp103/tp2.log`

13.4 Capture d'écran du fichier tp2.log

La figure suivante doit être remplacée par une capture d'écran réelle du fichier `rcp103/tp2.log`. Le fichier image peut être nommé :

`capture_tp2_log.png`



capture_tp2_log.png

FIGURE 1 – Capture d’écran du fichier rcp103/tp2.log

13.5 Analyse de la trace

La trace fournie contient :

```
1 2026-05-05 22:42:14 DEBUG __main__: +++ Engine : print_scheduler ...
2 2026-05-05 22:42:14 INFO __main__: Event ID: 1, Type: SEND_MSG, Time: 1.202
3 Event ID: 2, Type: RECV_MSG, Time: 1.916
4 Event ID: 3, Type: SEND_MSG, Time: 2.32
5 Event ID: 4, Type: RECV_MSG, Time: 2.391
6 Event ID: 5, Type: MSG_DEPT, Time: 4.572
7 Event ID: 6, Type: MSG_DEPT, Time: 5.916
```

Listing 33 – Extrait de tp2.log

Cette trace confirme que les événements sont affichés dans l'ordre chronologique. Elle valide donc le comportement attendu du scheduler.

14 Analyse critique

14.1 Points forts

Le travail réalisé présente plusieurs points forts :

- architecture orientée objet claire ;
- séparation entre interfaces et implémentations ;
- conventions de nommage lisibles ;
- tests séparés par composant ;
- usage du mocking ;
- configuration centralisée du logging ;
- trace exploitable pour vérifier l'ordre des événements.

14.2 Limites

Plusieurs limites peuvent être relevées :

- certains tests reposent encore principalement sur des affichages ;
- l'utilisation de chaînes comme "SEND_MSG" devrait être remplacée par `EventType.SEND_MSG` ;
- certains imports scientifiques sont présents avant d'être réellement utilisés ;
- les identifiants pourraient être rendus immuables ;
- les interfaces pourraient être davantage documentées.

14.3 Améliorations possibles

Les améliorations possibles sont :

- ajouter des assertions dans tous les tests ;
- créer un vrai test unitaire du scheduler ;
- normaliser l'usage de `EventType` ;
- ajouter des docstrings aux méthodes ;
- séparer les imports scientifiques dans des modules spécialisés ;
- améliorer la gestion du chemin absolu du fichier `tp2.log`.

15 Lien avec les séances suivantes

15.1 Séance 2

La séance 2 introduira les composants opérationnels :

- clients ;
- file d'attente ;
- serveur ;
- génération de messages ;
- traitement des arrivées et départs.

Les classes de la séance 1 seront directement réutilisées.

15.2 Séance 3

La séance 3 permettra d’assembler le simulateur complet :

- moteur de simulation ;
- passerelle ;
- métriques ;
- comparaison avec la théorie M/M/1 ;
- simulations M/M/1/K et M/M/c/K.

Les timestamps des messages permettront notamment de calculer :

$$W = \frac{1}{N} \sum_{i=1}^N (t_{\text{départ},i} - t_{\text{création},i})$$

La loi de Little pourra ensuite être utilisée :

$$L = \lambda W$$

16 Conclusion

Cette première séance a permis de construire les fondations du simulateur à événements discrets. Les classes `MessageImpl`, `EventImpl` et `SchedulerImpl` constituent les briques principales du modèle temporel. Le fichier `Engine.py` prépare le futur moteur de simulation, tandis que les tests et le logging permettent de valider progressivement le comportement du système.

D’un point de vue architectural, le projet suit une démarche cohérente : séparation interface/implémentation, tests par classe, conventions de nommage explicites et traçabilité par fichier de log. Ces choix facilitent la maintenance et préparent l’extension du simulateur lors des séances 2 et 3.

Les fichiers les plus représentatifs du travail réalisé sont donc :

- `MessageImpl.py` ;
- `EventImpl.py` ;
- `SchedulerImpl.py` ;
- `Engine.py`.

Ils constituent le cœur du rendu de la séance 1.

A Annexe A – Code de MessageImpl.py

```
1 from .IMessage import IMessage
2
3 class MessageImpl(IMessage):
4     """ Message envoy vers la passerelle """
5
6     message_id: int
7     source: str
8     destination: str
9     timestamp: float
10
11     def __init__(self, message_id, source, destination, timestamp=0.0):
12         self._message_id = message_id
13         self._source = source
14         self._destination = destination
15         self._timestamp = timestamp
16
17     ### getters du message ###
18     def get_message_id(self):
19         return self._message_id
20
21     def get_source(self):
22         return self._source
23
24     def get_destination(self):
25         return self._destination
26
27     def get_timestamp(self):
28         return self._timestamp
29
30     def set_message_id(self, message_id):
31         self._message_id = message_id
32
33     def set_source(self, source):
34         self._source = source
35
36     def set_destination(self, destination):
37         self._destination = destination
38
39     ### setters du message ###
40     def set_timestamp(self, temps):
41         self._timestamp = temps
42
43     # --- Affichage ---
44     def print_message(self):
45         return(f"[Message] ID={self._message_id} "
46              f"src={self._source} "
47              f"dst={self._destination} "
48              f"timestamp={self._timestamp:.4f}")
```

Listing 34 – MessageImpl.py

B Annexe B – Code de Message.py

```
1 from .IMessage import IMessage
2
3 class Message(IMessage):
4     """ Message envoy vers la passerelle """
5
6     message_id: int
7     source: str
8     destination: str
9     timestamp: float
10
11     def __init__(self, message_id, source, destination, timestamp=0.0):
12         self._message_id = message_id
```

```

13     self._source = source
14     self._destination = destination
15     self._timestamp = timestamp
16
17     ### getters du message ###
18     def get_message_id(self):
19         return self._message_id
20
21     def get_source(self):
22         return self._source
23
24     def get_destination(self):
25         return self._destination
26
27     def get_timestamp(self):
28         return self._timestamp
29
30     def set_message_id(self, message_id):
31         self._message_id = message_id
32
33     def set_source(self, source):
34         self._source = source
35
36     def set_destination(self, destination):
37         self._destination = destination
38
39     ### setters du message ###
40     def set_timestamp(self, temps):
41         self._timestamp = temps
42
43     # --- Affichage ---
44     def print_message(self):
45         message_str = (f"[Message] ID={self._message_id} "
46                       f"src={self._source} "
47                       f"dst={self._destination} "
48                       f"timestamp={self._timestamp:.4f}")
49         print(message_str)
50         return message_str

```

Listing 35 – Message.py

C Annexe C – Code de EventType.py

```

1 from enum import Enum
2
3 ## EventType      SEND_MSG, RECV_MSG, MSG_DEPT
4
5 class EventType(Enum):
6     SEND_MSG = 1
7     RECV_MSG = 2
8     MSG_DEPT = 3

```

Listing 36 – EventType.py

D Annexe D – Code de EventImpl.py

```

1 #!/usr/bin/python3
2 ## PYTHONPATH=. /usr/bin/python3 net/lecnam/rcp103/tp2/EventImpl.py
3 ## rapport      r diger: overleaf.com
4
5 #####
6 ## pre-req: in VSCode install extension
7 ## 'Microsoft Python Environments Extension'
8 ## ('Python Environment Manager' is deprecated)
9 ## https://scipy.org/install/: sudo apt-get install python3-scipy

```

```

10 ## test in shell with : pip list
11 ## Install on Windows:
12 ## c:\python314\python.exe -m pip install matplotlib
13 ## https://matplotlib.org/stable/install/index.html
14 ## sudo apt upgrade
15 ## sudo apt install python3-matplotlib
16 ## pip install matplotlib
17 ## python3 -m pip install -U pip
18 ## python3 -m pip install -U matplotlib
19 ## To manually trigger a refresh in VSCode:
20 ## Open the Command Palette (Cmd+Shift+P or Ctrl+Shift+P)
21 ## Run Python Environments: Refresh All Environment Managers
22
23 import datetime
24 import os
25 import platform
26 import string
27 import sys
28 import secrets
29 import traceback
30
31 import logging
32 import logging.config
33
34 from net.lecnam.rcp103.SimulateurException import SimulateurException
35
36 from scipy.stats import poisson
37 ## https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.poisson.html
38
39 import numpy as np
40 import matplotlib.pyplot as plt
41 import scipy
42
43 from net.lecnam.rcp103.tp2.IEvent import IEvent
44 from net.lecnam.rcp103.tp2.IMessage import IMessage
45 from net.lecnam.rcp103.tp2.EventType import EventType
46
47 ## print(scipy.__version__)
48 ## Always load logging_config.py from the same directory as this file
49
50 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")
51 logging.config.fileConfig(
52     config_path,
53     defaults=None,
54     disable_existing_loggers=True,
55     encoding=None
56 )
57
58 logger = logging.getLogger(__name__)
59
60 ## https://docs.python.org/3/library/logging.html#logging-levels
61 ## Class d'Implmentation
62
63 class EventImpl(IEvent):
64     # Implmentation de la classe EventImpl.
65     # Les methodes principales attendues sont :
66     # - constructeur
67     # - getters / setters
68     # - get_event_time()
69     # - print_event()
70     pass

```

Listing 37 – EventImpl.py

E Annexe E – Extrait de SchedulerImpl.py

```

1 ## Interface
2

```

```

3 from abc import ABC, abstractmethod
4 import datetime
5
6 from net.lecnam.rcp103.tp2.IEvent import IEvent
7 from net.lecnam.rcp103.tp2.IScheduler import IScheduler
8
9 class SchedulerImpl(IScheduler):
10     # Implémentation de l'ordonnanceur.
11     # Le r le attendu est de maintenir les vnements
12     # dans l'ordre chronologique.
13     pass

```

Listing 38 – SchedulerImpl.py

F Annexe F – Code de Engine.py

```

1 #!/usr/bin/python3
2 ## Commande pour lancer le programme :
3 ## PYTHONPATH=. python3 net/lecnam/rcp103/tp2/Engine.py
4 ## en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.Engine
5 ## sous Windows/PowerShell: python Engine.py
6
7 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
8 from net.lecnam.rcp103.tp2.EventImpl import EventImpl
9 from net.lecnam.rcp103.tp2.SchedulerImpl import SchedulerImpl
10
11 import logging
12 import logging.config
13 import os
14 import platform
15 import string
16 import sys
17
18 ## Always load logging_config.py from the same directory as this file
19
20 config_path = os.path.join(os.path.dirname(__file__), "logging_config.cnf")
21 logging.config.fileConfig(
22     config_path,
23     defaults=None,
24     disable_existing_loggers=True,
25     encoding=None
26 )
27
28 logger = logging.getLogger(__name__)
29
30 class Engine:
31     # Classe moteur de simulation.
32     # En s ance 1, elle sert principalement de point d'entr e
33     # et d'orchestrateur des tests.
34     pass
35
36 if __name__ == "__main__":
37     engine = Engine()
38     engine.create_clients(3)
39     engine.run_tests()

```

Listing 39 – Engine.py

G Annexe G – Code de MessageImplTest.py

```

1 #!/usr/bin/python3
2 ## Commande pour lancer le programme :
3 ## PYTHONPATH=. python3 net/lecnam/rcp103/tp2/MessageImplTest.py
4 ## en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.MessageImplTest
5 ## sous Windows/PowerShell: python MessageImplTest.py

```

```

6
7 from datetime import datetime
8 import traceback
9 import logging
10
11 from net.lecnam.rcp103.tp2.EventImpl import EventImpl
12 from net.lecnam.rcp103.tp2.IEvent import IEvent
13 from net.lecnam.rcp103.tp2.IMessage import IMessage
14 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
15
16 try:
17     print("+++ START MessageImplTest fire")
18
19     impl = MessageImpl(1, "Alice", "Bob", 1.21)
20     src = impl.get_source()
21     dst = impl.get_destination()
22
23     print(f"Source: {src}, Destination: {dst}")
24
25     res = impl.print_message()
26     print("+++ Pretty print = " + str(res))
27
28     print("+++ END MessageImplTest ignite")
29
30 except RuntimeError as error:
31     print("Erreur au Runtime dans MessageImplTest")
32     print(f"A {type(error).__name__} has occurred.")
33     exit(42)
34
35 except Exception as exception:
36     print("Exception dans MessageImplTest/Main")
37     print(f"A {type(exception).__name__} has occurred.")
38     print()
39     traceback.print_exc()
40     print()
41     print(exception)
42     exit(42)

```

Listing 40 – MessageImplTest.py

H Annexe H – Code de EventImplTest.py

```

1 #!/usr/bin/python3
2 ## Commande pour lancer le programme :
3 ## PYTHONPATH=. python3 net/lecnam/rcp103/tp2/EventImplTest.py
4 ## en Linux/WSL: python3 -m net.lecnam.rcp103.tp2.EventImplTest
5 ## sous Windows/PowerShell: python EventImplTest.py
6
7 from datetime import datetime
8 import traceback
9 import logging
10
11 from net.lecnam.rcp103.tp2.EventImpl import EventImpl
12 from net.lecnam.rcp103.tp2.IEvent import IEvent
13 from net.lecnam.rcp103.tp2.IMessage import IMessage
14 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
15 from net.lecnam.rcp103.tp2.EventType import EventType
16
17 try:
18     print("+++ START EventImpl fire")
19
20     msg = MessageImpl(1, "Alice", "Bob", 1.13)
21     impl = EventImpl(1, msg, "SEND_MSG", 1.13)
22
23     horodatage = impl.get_event_time()
24     print("+++ result = " + str(horodatage))
25
26     res = impl.print_event()

```



```

27     print("+++ pretty print = " + str(res))
28
29     print("+++ END EventImpl ignite")
30
31 except RuntimeError as error:
32     print("Erreur au Runtime dans EventImpl")
33     print(f"A {type(error).__name__} has occurred.")
34     exit(42)
35
36 except Exception as exception:
37     print("Exception dans EventImpl/Main")
38     print(f"A {type(exception).__name__} has occurred.")
39     print()
40     traceback.print_exc()
41     print()
42     print(exception)
43     exit(42)

```

Listing 41 – EventImplTest.py

I Annexe I – Code de EventImplMockTest.py

```

1  #!/usr/bin/python3
2  ## PYTHONPATH=. python3 net/lecnam/rcp103/tp2/EventImplMockTest.py
3  ## Use unittest.mock to create a mock SimulateurImpl for Mock testing
4  ## https://docs.python.org/3/library/unittest.mock.html
5
6  import traceback
7  import logging
8
9  from unittest.mock import MagicMock
10 from net.lecnam.rcp103.tp2.EventImpl import EventImplMockTest
11
12 from net.lecnam.rcp103.tp2.EventImpl import EventImpl
13 from net.lecnam.rcp103.tp2.IEvent import IEvent
14 from .IMessage import IMessage
15 from net.lecnam.rcp103.tp2.MessageImpl import MessageImpl
16 from net.lecnam.rcp103.tp2.EventType import EventType
17
18 ## Create a mock instance of SimulateurImpl
19
20 mock_sim = MagicMock(spec=EventImpl)
21
22 ## Example: set return value for calcul
23
24 mock_sim.get_event_time.return_value = 1.42
25
26 msg = MessageImpl(1, "Test message", "Alice", "Bob")
27
28 ## Use the mock in your test
29
30 result = mock_sim.get_event_time()
31
32 print("Mocked calcul result:", result)
33
34 ## You can also assert calls
35
36 mock_sim.get_event_time.assert_called_with()

```

Listing 42 – EventImplMockTest.py

J Annexe J – Code de logging_config.cnf

```

1 [loggers]
2 keys=root

```

```

3
4 [handlers]
5 keys=consoleHandler,fileHandler
6
7 [formatters]
8 keys=simpleFormatter
9
10 [logger_root]
11 level=DEBUG
12 handlers=consoleHandler,fileHandler
13
14 [handler_consoleHandler]
15 class=StreamHandler
16 level=DEBUG
17 formatter=simpleFormatter
18 args=(sys.stdout,)
19
20 [handler_fileHandler]
21 class=FileHandler
22 level=DEBUG
23 formatter=simpleFormatter
24 args=('tp2.log', 'a')
25
26 [formatter_simpleFormatter]
27 format=%(asctime)s %(levelname)s %(name)s: %(message)s
28 datefmt=%Y-%m-%d %H:%M:%S

```

Listing 43 – *logging_config.cnf*

K Annexe K – Extrait du fichier tp2.log

```

1 2026-05-05 22:42:14 DEBUG __main__: +++ Engine : print_scheduler ...
2 2026-05-05 22:42:14 INFO __main__: Event ID: 1, Type: SEND_MSG, Time: 1.202
3 Event ID: 2, Type: RECV_MSG, Time: 1.916
4 Event ID: 3, Type: SEND_MSG, Time: 2.32
5 Event ID: 4, Type: RECV_MSG, Time: 2.391
6 Event ID: 5, Type: MSG_DEPT, Time: 4.572
7 Event ID: 6, Type: MSG_DEPT, Time: 5.916

```

Listing 44 – *tp2.log*